

Symmetric Authenticated Tunneling Protocol (SATP)

Revision 1.0a, March 14, 2025

John G. Underhill – john.underhill@protonmail.com

This document is an engineering level description of the SATP authenticated and encrypted remote user verification and tunneling protocol. This document describes SATP as a symmetric-only authenticated tunnel using deterministic key provisioning, a two-message tunnel establishment exchange, authenticated packet encryption, and a post-tunnel client authentication callback that may be backed by SCB-verified credentials.

Contents	Page
<u>Foreword</u>	2
<u>1: Introduction</u>	3
<u>2: Scope</u>	5
<u>3: Terms and Definitions</u>	6
<u>4: Cryptographic Primitives</u>	9
<u>5: Protocol Description</u>	11
<u>6: Mathematical Description</u>	24
<u>7: Security Analysis</u>	30
<u>8: Use Case Scenarios</u>	35
<u>Conclusion</u>	38

Foreword

This document is intended as the preliminary draft of a new standards proposal, and as a basis from which that standard can be implemented. We intend that this serves as an explanation of this new technology, and as a complete description of the protocol.

This document is the first revision of the specification of SATP, further revisions may become necessary during the pursuit of a standard model, and revision numbers shall be incremented with changes to the specification. The reader is asked to consider only the most recent revision of this draft, as the authoritative implementation of the SATP specification.

The author of this specification is John G. Underhill, and can be reached at john.underhill@protonmail.com

SATP, the algorithm constituting the SATP messaging protocol is patent pending, and is owned by John G. Underhill and Quantum Resistant Cryptographic Solutions Corporation. The code described herein is copyrighted, and owned by John G. Underhill and Quantum Resistant Cryptographic Solutions Corporation.

1. Introduction

Key distribution remains one of the most challenging problems in cryptography. The internet has rapidly evolved into a critical communications platform, used daily by billions of people worldwide. Given its foundational role in global commerce, personal communication, and critical infrastructure, information transmitted over the internet must be strongly protected from interception, tampering, or future compromise.

Currently, the predominant security model relies heavily on asymmetric cryptography (public/private key cryptography) to establish secure encrypted tunnels and authenticate entities. These asymmetric primitives depend on mathematical ‘trapdoor’ functions, problems easily created with a public key but computationally infeasible to solve without the corresponding private key. However, asymmetric cryptographic methods have always carried an inherent risk: their security assumptions can be undermined over time by breakthroughs in mathematics or advances in computing technology.

The emergence of quantum computing is now demonstrating this risk in a concrete and alarming manner. Quantum computers, leveraging quantum mechanics, threaten to render entire classes of asymmetric algorithms, such as RSA and elliptic-curve cryptography (ECC) obsolete within a foreseeable timeframe. Intelligence agencies already capture and archive encrypted data streams on a massive scale, aware that today's secure communications could eventually become readable if future computational breakthroughs materialize. Even newer asymmetric systems, such as lattice-based cryptography, while resistant to currently known quantum attacks, may face similar threats as quantum computing and mathematical techniques mature.

In addition, parameter selection for asymmetric cryptography often involves trade-offs, choosing more modest security margins in exchange for improved performance. This compromise may accelerate the vulnerability of encrypted communications to future cryptanalysis. Considering that truly secure communication may need to remain confidential over decades, possibly spanning entire human lifetimes, the inherent unpredictability of asymmetric cryptography's future safety becomes unacceptable.

Symmetric cryptography presents an attractive alternative or complements to asymmetric methods. Provided that sufficiently strong symmetric cryptographic primitives and adequately long key lengths are chosen, symmetric encryption can offer formidable protection against future threats, potentially resisting cryptanalysis indefinitely. Nevertheless, symmetric key-based systems historically struggle with challenges like scalability, key distribution complexity, forward secrecy, and vulnerability to single points of failure.

For example, traditional symmetric schemes relying upon pre-shared keys often use a single fixed key and session counters to derive encryption keys. Such approaches, seen in early SSH implementations, exhibit critical security weaknesses: capturing a single host's secret key can compromise all past communications from that host; compromising a server's key storage can potentially expose past, present, and future communication streams for all network participants.

The Symmetric Authenticated Tunneling Protocol (SATP) presented in this paper proposes a symmetric solution designed to reduce dependence on online asymmetric key establishment. SATP employs securely provisioned, hierarchical symmetric key derivation, using keys stored securely on client devices, such as SIM, Micro-SD, USB, embedded memory, or secure element devices. It generates per-session channel keys from a one-time client leaf key and a fresh client nonce. This provides single-use leaf separation and future-key resistance, provided that client storage is advanced atomically and consumed leaf keys are erased before reuse.

SATP thereby aims to provide a compact post-quantum-oriented authenticated tunnel for environments in which symmetric provisioning, deterministic auditability, and low-latency establishment are preferred over certificate-based public-key exchange. Its security depends on protection of provisioned symmetric material, uniqueness of consumed key indices, nonce uniqueness, and the security of the underlying RCS, cSHAKE, KMAC, SHA-3, and SCB constructions.

1.1 Purpose

The SATP secure messaging protocol, utilized in conjunction with quantum secure symmetric cryptographic primitives, is used to create an encrypted and authenticated duplexed communications channel. This specification presents a secure messaging protocol that creates an encrypted communications channel, in such a way that:

- 1) The symmetric cipher keys for the send and receive channels are per-session values derived from a single-use device leaf key and a fresh client nonce.
- 2) The capture of a later unused client leaf does not directly reveal information about earlier sessions whose leaf keys were consumed and erased, assuming the long-term branch/server secrets and recorded active leaf material were not compromised at the time of those sessions.
- 3) SATP provides server authentication during tunnel establishment through a validation hash bound to the device leaf, client nonce, and long-term server secret, followed by client authentication over the encrypted tunnel through an application-supplied authentication callback.

SATP is a duplexed communications system. Symmetric cipher keys are generated for each session. The system works in a client/server model, where a client requests a connection from the server and initiates the key exchange. These keys initialize authenticated symmetric cipher states for both communication directions. The SATP packet header is authenticated as associated data for encrypted packets, binding the packet flag, payload length, sequence number, and UTC timestamp to the encrypted message.

2. Scope

This document describes the SATP secure messaging protocol, which is used to establish an encrypted and authenticated duplexed message stream between two hosts. This document describes the implemented symmetric key hierarchy, the two-message tunnel establishment exchange, post-tunnel client authentication, packet encryption, packet authentication, replay checks, and operational constraints of the reference implementation.

2.1 Application

This protocol is intended for institutions that implement secure communication channels used to encrypt and authenticate secret information exchanged between remote terminals.

The key exchange functions, authentication and encryption of messages, and message exchanges between terminals defined in this document must be considered as mandatory elements in the construction of an SATP communications stream. Components that are not necessarily mandatory, but are the recommended settings or usage of the protocol shall be denoted by the key-words **SHOULD**. In circumstances where strict conformance to implementation procedures is required but not necessarily obvious, the key-word **SHALL** will be used to indicate compulsory compliance is required to conform to the specification.

3. Terms and Definitions

3.1 Cryptographic Primitives

3.1.1 SCB

The SHAKE Cost Based Key Derivation Function (SCB-KDF) uses advanced techniques such as cache thrashing, memory ballooning, and a CPU intensive core function to mitigate attacks on a hash function by making it more expensive to run dictionary and rainbow attacks to discover a user's passphrase.

3.1.2 RCS

The wide-block Rijndael hybrid authenticated AEAD symmetric stream cipher.

3.1.3 SHA-3

The SHA3 hash function NIST standard, as defined in the NIST standards document FIPS-202; SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.

3.1.4 SHAKE

The NIST standard Extended Output Function (XOF) defined in the SHA-3 standard publication FIPS-202; SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.

3.1.5 KMAC

The SHA3 derived Message Authentication Code generator (MAC) function defined in NIST special publication SP800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash and ParallelHash.

3.2 Network References

3.2.1 Bandwidth

The maximum rate of data transfer across a given path, measured in bits per second (bps).

3.2.2 Byte

Eight bits of data, represented as an unsigned integer ranged 0-255.

3.2.3 Certificate

A digital certificate, a structure that contains a signature verification key, expiration time, and serial number and other identifying information. A certificate is used to verify the authenticity of a message signed with an asymmetric signature scheme.

3.2.4 Domain

A virtual grouping of devices under the same authoritative control that shares resources between members. Domains are not constrained to an IP subnet or physical location but are a virtual

group of devices, with server resources typically under the control of a network administrator, and clients accessing those resources from different networks or locations.

3.2.5 Duplex

The ability of a communication system to transmit and receive data; half-duplex allows one direction at a time, while full-duplex allows simultaneous two-way communication.

3.2.6 Gateway: A network point that acts as an entrance to another network, often connecting a local network to the internet.

3.2.7 IP Address

A unique numerical label assigned to each device connected to a network that uses the Internet Protocol for communication.

3.2.8 IPv4 (Internet Protocol version 4): The fourth version of the Internet Protocol, using 32-bit addresses to identify devices on a network.

3.2.9 IPv6 (Internet Protocol version 6): The most recent version of the Internet Protocol, using 128-bit addresses to overcome IPv4 address exhaustion.

3.2.10 LAN (Local Area Network)

A network that connects computers within a limited area such as a residence, school, or office building.

3.2.11 Latency

The time it takes for a data packet to move from source to destination, affecting the speed and performance of a network.

3.2.12 Network Topology

The arrangement of different elements (links, nodes) of a computer network, including physical and logical aspects.

3.2.13 Packet

A unit of data transmitted over a network, containing both control information and user data.

3.2.14 Protocol

A set of rules governing the exchange or transmission of data between devices.

3.2.15 TCP/IP (Transmission Control Protocol/Internet Protocol)

A suite of communication protocols used to interconnect network devices on the internet.

3.2.16 Throughput: The actual rate at which data is successfully transferred over a communication channel.

3.2.17 UDP (User Datagram Protocol)

A communication protocol that offers a limited amount of service when messages are exchanged between computers in a network that uses the Internet Protocol.

3.2.18 VLAN (Virtual Local Area Network)

A logical grouping of network devices that appear to be on the same LAN regardless of their physical location.

3.2.19 VPN (Virtual Private Network)

Creates a secure network connection over a public network such as the internet.

3.3 Normative References

The following documents serve as references for cryptographic components used by SATP:

3.3.1 FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable Output Functions:

This standard specifies the SHA-3 family of hash functions, including SHAKE extendable-output functions. <https://doi.org/10.6028/NIST.FIPS.202>

3.3.2 NIST SP 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash:

This publication specifies four SHA-3-derived functions: cSHAKE, KMAC, TupleHash, and ParallelHash. <https://doi.org/10.6028/NIST.SP.800-185>

3.3.3 NIST SP 800-90A Rev. 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators:

This publication provides recommendations for the generation of random numbers using deterministic random bit generators. <https://doi.org/10.6028/NIST.SP.800-90Ar1>

3.3.4 NIST SP 800-108: Recommendation for Key Derivation Using Pseudorandom Functions:

This publication offers recommendations for key derivation using pseudorandom functions. <https://doi.org/10.6028/NIST.SP.800-108>

4. Cryptographic Primitives

SATP relies on a robust set of symmetric cryptographic primitives designed to provide resilience against both classical and quantum-based attacks. The following sections detail the specific cryptographic algorithms and mechanisms that form the foundation of SATP's encryption, key exchange, and authentication processes.

4.1 Symmetric Cryptographic Primitives

SATP's symmetric encryption employs the Rijndael Cryptographic Stream (RCS), a stream cipher adapted from the Rijndael (AES) algorithm to meet post-quantum security needs. Key features of the RCS cipher include:

- **Wide-Block Cipher Design:** RCS extends the original AES design with a focus on increasing the block size and number of transformation rounds, thereby enhancing its resistance to differential and linear cryptanalysis.
- **Enhanced Key Schedule:** The key schedule in RCS is cryptographically strengthened using Keccak, ensuring that derived keys are resistant to known attacks, including algebraic-based and differential attacks.
- **Authenticated Encryption with Associated Data (AEAD):** RCS integrates with KMAC (Keccak-based Message Authentication Code) to provide both encryption and message authentication in a single operation. This approach ensures that data integrity is maintained alongside confidentiality.

The RCS stream cipher's design is optimized for high-performance environments, making it suitable for low-latency applications that require secure and efficient data encryption. It leverages AVX/AVX2/AVX512 intrinsics and AES-NI instructions embedded in modern CPUs.

4.2 Hash Functions and Key Derivation

Hash functions and key derivation functions (KDFs) are essential to SATP's ability to transform raw cryptographic data into secure keys and hashes. The following primitives are used:

- **SHA-3:** SHA-3 serves as SATP's primary hash function, providing secure, collision-resistant hashing capabilities.
- **SHAKE and cSHAKE:** SATP employs Keccak-derived XOF functions for deriving symmetric keys from shared secrets. The reference implementation uses cSHAKE-256 with explicit key, function-name, and customization inputs rather than treating all inputs as an undifferentiated concatenation.

These cryptographic primitives ensure that SATP's key management processes remain secure, even in scenarios involving high-risk adversaries and quantum-capable threats.

4.3 Key Derivation Function

SCB is a cost-based key derivation function, one that can increase the memory usage and computational complexity of the underlying hash function. Suitable for password hashing, key generation, and in cases where brute-force attacks on a derived key must be strongly mitigated.

The SCB KDF's architecture is designed to withstand a variety of cryptographic attacks by enforcing both computational and memory hardness.

- **Brute-Force Attacks:** The high computational and memory costs imposed by SCB exponentially increase the time required to test each key, rendering brute-force attacks infeasible within practical timeframes.
- **Dictionary Attacks:** Memory-hardness ensures that generating and storing comprehensive dictionaries would require exorbitant memory resources, making such attacks impractical.
- **Rainbow Table Attacks:** The iterative and memory-intensive nature of SCB disrupts the feasibility of creating effective rainbow tables, as each table entry would necessitate substantial memory resources and computational effort.
- **Side-Channel Attacks:** The deterministic scattering pattern and uniform memory access intervals obscure access patterns, minimizing timing discrepancies and reducing information leakage through side channels.
- **Parallelized Hardware Attacks:** Each write of a cache line to a memory location, writes the cache position index and loop iterator to the key hash, and the entire buffer is written to the hash at each L2 sized interval (default) 256 KiB, sequential operations that make any significant parallelization impossible.

The SCB KDF's architecture is designed to withstand a variety of cryptographic attacks by enforcing both computational and memory hardness.

5. Protocol Description

5.1 Structures

5.1.1 Device Key Set

The device key is an internal structure that stores the device identity and current key index, the long-term server secret, the expiration value, and the fixed-size client key tree.

Parameter	Data Type	Bit Length	Function
Expiration	Uint64	64	Provisioning validity value
KID	Uint8 array	128	Device identity and key index
STc	Uint8 array	256	Long-term server secret used for server validation
Kidx	Uint32	32	Key index encoded in the final four bytes of KID
Ktree	Uint8 array	256 * 1024	Device key tree; 1024 leaves

Table 5.1.1a: The client key structure.

The expiration parameter is a 64-bit unsigned integer that stores a provisioning validity value. The reference implementation stores and serializes this value with master, server, and device keys. Time-based rejection of expired provisioning records is a deployment policy requirement and is not performed by the two-message key exchange path.

The key identity array is a 16-byte array that uniquely identifies a client device key. This identifier can be used to match the key on a branch server. The key identity array, is divided into subsections, identification numbers for the master key, branch key, epoch class, service ID, device key, and key index.

Master ID	Branch ID	Epoch Class	Service ID	Device ID	Key ID
2 bytes	2 bytes	2 bytes	2 bytes	4 bytes	4 bytes

Table 5.1.1b: The device identity structure.

The master root key is processed with the branch/server identity to derive a server branch key. The reference implementation supports 65,535 possible domain identifiers and 65,535 possible branch identifiers within the two-byte domain and two-byte branch fields.

The server branch key is processed with the full 16-byte key identity string to derive device key leaves. In the reference implementation the generated device key tree contains SATP_KEY_TREE_COUNT leaves, where SATP_KEY_TREE_COUNT is 1024. The final four bytes of the KID encode the current key index in big-endian order, and valid connection attempts use an index lower than SATP_KEY_TREE_COUNT.

The epoch class is incremented whenever you roll a new root, branch, or algorithm suite, and is used for bulk revocation and migration. A server can reject any identity whose Epoch $\neq$ the current epoch, instantly disabling whole SIM batches after a root-key leak.

The service class selects one of up to 65,535 access profiles (e.g., “VPN”, “IoT-uplink”, “High-priv-admin”), and is used for fine-grained authorization. The same user can present different classes to access different services, all while the rest of the identity string remains unchanged.

5.1.2 Branch Key

The server branch key is the upstream derivation key used to regenerate client device leaves on demand. The reference implementation derives the server derivation key from the master key using cSHAKE-256 with SATP_CONFIG_STRING as the function-name input and the 4-byte server identity as the customization input. Device leaves are derived from the server derivation key and the 16-byte KID.

The branch key is itself a derivation of the 4-byte server identity, consisting of a 16-bit domain identity and a 16-bit branch identity, processed with the master root derivation key.

Clients loaded by a branch server inherit the expiration time field, the seconds from epoch that define the time the keys are valid. This branch expiration time is inherited from the master root key.

Parameter	Data Type	Bit Length	Function
Expiration	UInt64	64	Validity check
SID	UInt8 array	32	Identification
SDK	UInt8 array	256	Derivation Key

Table 5.1.2: The server key structure.

5.1.3 Root Key

The master root key is identical to the branch keys except for the bit length of the key identification array is a total sixteen bits, whereas the branch identities are combined with the domain root identity string. The root master key structure contains the domains master key, which when hashed along with the root and branch identity strings, produces unique branch keys for every SATP server in the domain.

Parameter	Data Type	Bit Length	Function
Expiration	Uint64	64	Validity check
MID	Uint8 array	16	Identification
MDK	Uint8 array	256	Derivation Key

Table 5.1.3: The master key structure.

5.1.4 Client State

The client state is an internal structure that contains the variables required by SATP operations. This includes elements copied from the client key structure during initialization, send and receive channel cipher states, the server-validation hash, packet counters, and the exchange state flag.

Data Name	Data Type	Bit Length	Function
Exp	Uint64	64	Key Expiration Time
Kidx	Uint32	32	Key Tree Index
Ktree	Uint8 array	$256 * n$	The Key Tree
STc	Uint8 array	256	The Server Salt
Hc	Uint8 array	256	Session Hash
Cid	Uint8 array	128	Client Identification
RXseq	Uint64	64	Packet Counter
TXseq	Uint64	64	Packet Counter
Cipher Receive State	Structure	Variable	Symmetric Decryption
Cipher Transmit State	Structure	Variable	Symmetric Encryption
ExFlag	Uint8	8	Protocol Status

Table 5.1.4: The client state structure.

5.1.5 Branch State

The branch server state is similar to the client state, except that it stores the server derivation key rather than a client key tree. The server regenerates the active device leaf from the received KID and uses the long-term server secret to compute the server-validation hash Hc.

Data Name	Data Type	Bit Length	Function
-----------	-----------	------------	----------

Exp	Uint64	64	Key Expiration Time
Kbr	Uint8 array	256	Derivation Key
Hc	Uint8 array	256	Session Hash
STc	Uint8 array	256	The Server Salt
Sp	Uint8 array	256	Session Hash
Dbr	Uint8 array	32	Server Identification
RXseq	Uint64	64	Packet Counter
TXseq	Uint64	64	Packet Counter
Cipher Receive State	Structure	Variable	Symmetric Decryption
Cipher Transmit State	Structure	Variable	Symmetric Encryption
ExFlag	Uint8	8	Protocol Status

Table 5.1.5: The branch state structure.

5.1.6 Root State

The root contains the master domain key (K_{root}), the expiration time that applies for all derived keys (*exp*), and the root identification string.

Data Name	Data Type	Bit Length	Function
Exp	Uint64	64	Key Expiration Time
Kroot	Uint8 array	256	Derivation Key
Ddom	Uint8 array	16	Root Identification

Table 5.1.6: The root state structure.

5.1.7 Reserved Keep Alive Constants

Parameter	Data Type	Bit Length	Function
Ktime	Uint64	64	Reserved; not used by active SATP v1.0a packet flow
Pseq	Uint64	64	Reserved; not used by active SATP v1.0a packet flow
Rstat	Bool	8	Reserved; not used by active SATP v1.0a packet flow

Table 5.1.7: Reserved keep-alive fields. The active SATP v1.0a exchange does not define or transmit keep-alive packets. These fields and constants are not normative for the implemented tunnel and are retained only as reserved implementation artifacts.

5.1.8 SATP Packet Header

The SATP packet header is 21 bytes in length. Multi-byte integer fields in the packet header are serialized in little-endian order. The on-wire field order is:

1. The **Packet Flag**, the type of message contained in the packet; this can be any one of the key-exchange stage flags, a message, or an error flag.
2. The **Message Size**, a 32-bit unsigned little-endian value that gives the encrypted or plaintext message payload length in bytes.
3. The **Packet Sequence**, a 64-bit unsigned little-endian sequence value used for strict in-order packet processing.
4. The **Packet Creation time**, a 64-bit unsigned little-endian UTC timestamp in seconds from the implementation epoch.

The message is a variable sized array, up to SATP_MESSAGE_MAX in size.

Packet Flag 1 byte	Message Size 4 bytes	Packet Sequence 8 bytes	Packet Creation 8 bytes
Message Variable Size			

Table 5.1.8: The SATP packet structure.

This packet structure is used for both the key exchange protocol and the encrypted tunnel. For encrypted packets the serialized header is supplied to the cipher as associated data before encryption or decryption.

5.1.9 Flag Types

The following are a preliminary list of packet flag enumeration types used by SATP:

Flag Name	Hex Value	Flag Purpose
None	0x00	No flag was specified, the default value.
Connect Request	0x01	The client connection request packet.
Connect Response	0x02	The server connection response packet.
Connection Terminated	0x03	A connection termination notification.
Encrypted Message	0x04	An encrypted tunnel message. The implemented client-authentication request and response are carried under this flag.

Authentication Request	0x05	Reserved authentication request flag; not emitted by the current reference implementation.
Authentication Response	0x06	Reserved authentication response flag; not emitted by the current reference implementation.
Authentication Verify	0x08	Reserved authentication verification flag; not emitted by the current reference implementation.
Keep Alive Request	0x09	Reserved keep-alive flag; no active keep-alive exchange is defined in SATP v1.0a.
Session Established	0x0A	Internal session state value; not emitted as a wire packet by the current reference implementation.
Error Condition	0xFF	The packet contains an error condition.

Table 5.1.9: Packet header flag types.

5.1.10 Error Types

The following are a preliminary list of error messages used by SATP:

Error Name	Hex Value	Description
None	0x00	No error condition was detected.
Accept Fail	0x01	The socket accept operation failed.
Authentication Failure	0x02	The client authentication failed.
Authentication Success	0x03	The client authentication succeeded.
Listen Fail	0x04	The listener socket could not connect.
Allocation Failure	0x05	Memory could not be allocated.
Bad Keep Alive	0x06	Reserved keep-alive error code.
Cipher Auth Failure	0x07	The cipher authentication failed.
Channel Down	0x08	The communication channel failed.
Connection Failure	0x09	The device could not connect to the remote host.
Decryption Failure	0x0A	Authenticated decryption failed.
Device Unrecognized	0x0B	The device identity is unrecognized.
Establish Failure	0x0C	The establishment phase failed.
Hosts Exceeded	0x0D	The server has no available connection slots.
Invalid Input	0x0E	The input provided is invalid.
Invalid Request	0x0F	The packet flag or request was unexpected.

Keep Alive Expired	0x10	Reserved keep-alive expiration code.
KEX Auth Failure	0x11	The key exchange authentication failed.
Key Expired	0x12	The key has expired.
Key Not Recognized	0x13	The key identity is not recognized.
Listener Fail	0x14	The listener failed to initialize.
Message Time Invalid	0x15	The packet timestamp is outside the validity window.
Packet Expired	0x16	The packet validity check failed.
Packet Unsequenced	0x17	The packet was received out of sequence.
Random Failure	0x18	The random generator failed.
Receive Failure	0x19	The receiver failed at the network layer.
Transmit Failure	0x1A	The transmitter failed at the network layer.
Unknown Protocol	0x1B	The protocol string was not recognized.
Unsequenced	0x1C	The packet sequence number is out of sync.
Verify Failure	0x1D	The expected data could not be verified.
Disconnect Request	0x1E	The remote host disconnected.
General Failure	0x1F	A general failure occurred.

Table 5.1.10: Error type messages.

5.2 Operational Overview

In SATP's multi-tiered hierarchical topology, a master root key (K_{root}) is generated first, serving as the foundational key. The reference implementation uses this master key with a 4-byte server identity to derive a branch/server derivation key by cSHAKE-256. The branch/server key is then used to generate and regenerate client device leaves.

The server generates symmetric key batches for client devices associated with a branch. These keys are securely provisioned onto devices such as secure SIM cards, USB tokens, embedded secure memory, or equivalent protected storage. The reference implementation serializes a device record containing the KID, STc, expiration value, and a fixed 1024-leaf key tree. Each individual key leaf within the SATP framework is associated with a unique 16-byte KID and is intended to be consumed exactly once by the client. The supplied client application advances the serialized device-key file before tunnel use by erasing the active leaf and incrementing the key index. If persistent storage is not advanced atomically, leaf reuse can occur after a crash or power loss.

The expiration or renewal cycle for keys depends strongly upon the target application. For example, financial applications might tolerate relatively infrequent key rotations, whereas high-security communications scenarios may require shorter renewal intervals, possibly daily or weekly, to ensure maximal security.

It is recommended that branch-level keys (K_{br}) and individual client keys ($K_{c,i}$) be periodically refreshed. This key rotation process ensures ongoing resilience against scenarios where either branch-level secrets or client-held key material is compromised. Entropy can be introduced periodically using a strong, post-quantum secure encrypted tunnel protocol such as QSTP, to inject fresh randomness into branch or device keys. By mixing new entropy with the existing embedded keys, the protocol could periodically generate refreshed base keys that are fully

independent from prior key material. These refreshed keys can be distinguished by an epoch identifier (E_{pc}) embedded within each key identity string, marking each new key revision clearly.

Any protocol error encountered during session establishment, key exchange, or normal tunnel communications triggers immediate session termination. Either the client or server will send an explicit error notifications to their peer, disconnect gracefully, and securely erase ephemeral session state. Protocol errors include but are not limited to message synchronization mismatches, unexpected message sizes during exchanges, authentication or verification failures, and cryptographic or networking function errors.

SATP incorporates replay controls for the established encrypted tunnel. Each encrypted packet contains a 64-bit timestamp and a 64-bit sequence number. The recipient verifies that the timestamp is within the configured validity window and that the sequence number is exactly the next expected value. The reference implementation therefore enforces strict in-order packet processing rather than a sliding replay window.

During tunnel establishment and encrypted transport, serialized packet headers are used as authenticated associated data where encryption is applied. This binds the packet flag, message size, sequence number, and timestamp to the protected message. The plaintext connect request is validated by flag, sequence, message length, and timestamp before the server derives the response state.

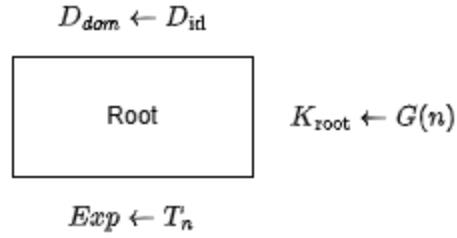
5.2.1 Root Initialization

A root server can be implemented in a large distributed network. The root server is optional though; a branch server can be implemented as a standalone server, in which case the domain string (representing the 16-bit root domain identifier) can be merged with the branch identifier creating a single 32-bit server identity string.

The domain identity describes the unique domain that the network resides in, and is the root of the server network identification.

The root server stores the domains master derivation key K_{root} . This key is used as the root of the key derivation tree, when hashed with the unique domain and branch identity strings, and when those unique branch keys are hashed with the client and key identity strings to create the client hash tree.

The expiration field is a 64-bit unsigned integer value representing the seconds from the epoch (01/01/1900) that the root key expires.



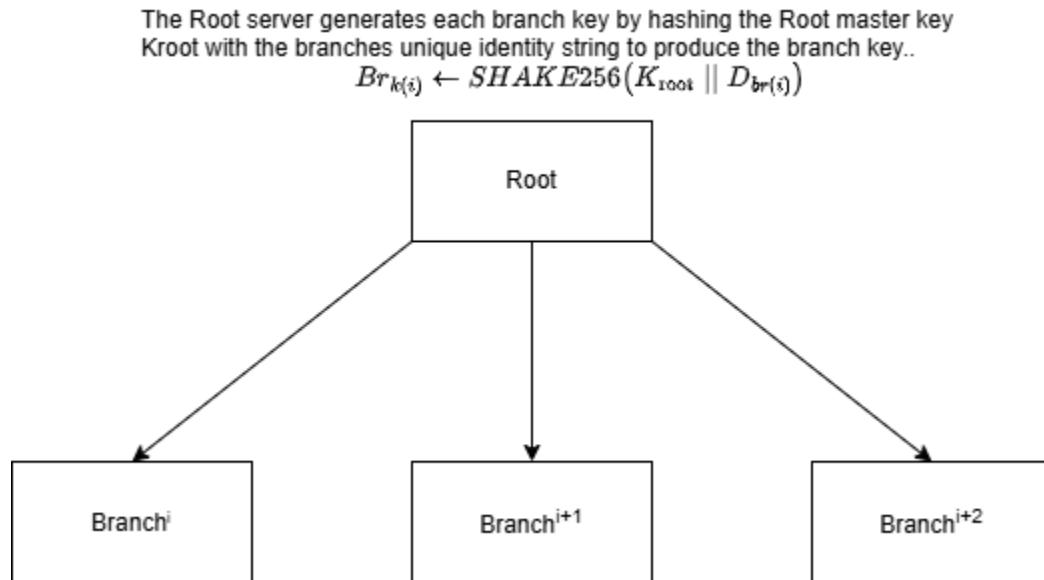
The 16-bit Root domain identity D_{id} is created, the key expiration time Exp is set, and a master domain key K_{root} is generated.

The expiration time is set, the master derivation key is generated using a random provider, and the domain bits are set, to create the root domain key structure.

5.2.2 Branch Initialization

The branch server is initialized by deriving the server branch key from the master derivation key and the 4-byte server identity. The server identity is formed from the 16-bit domain identifier and the 16-bit branch identifier. The reference implementation uses cSHAKE-256 with SATP_CONFIG_STRING as the function-name input and the server identity as customization input.

The root is used to generate the branch key structures, which are loaded onto branch servers in the domain.



Once the branch servers have had their key structures initialized, they can be used to assign client identities and create client key trees.

5.2.3 Client Initialization

The branch server assigns each client a unique 32-bit client identification string, this is appended to the domain and branch server string, to form the entire *root\branch\client* identity.

The branch hashes the branch key with the full client identity string and a monotonic counter encoded in the final four bytes of the KID. The reference implementation pre-computes SATP_KEY_TREE_COUNT device leaves, where SATP_KEY_TREE_COUNT is 1024. The current key index is stored as a 32-bit unsigned integer in big-endian order within the KID. The implementation accepts indices from 0 through 1023 inclusive.

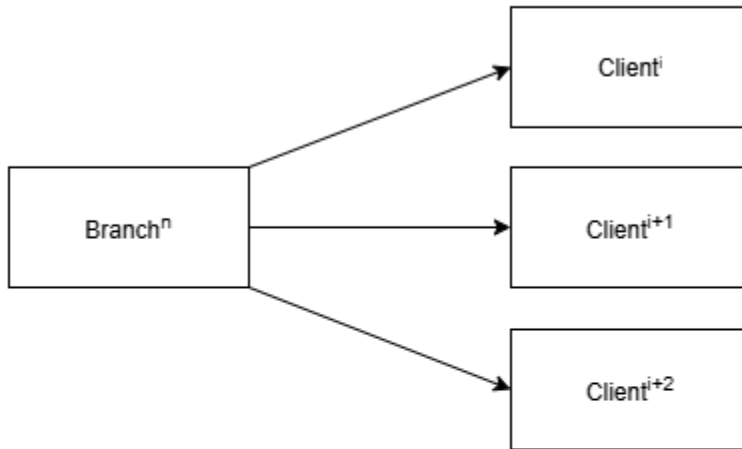
The branch hashes the branch key along with the full client identity string and an incrementing monotonic counter that represents the index number of each key created for the client's key-tree. The key-tree; the set of ephemeral keys assigned to the client, reveals no knowledge of the branch key to the client, and can be of any count up to the SATP_KEY_TREE_MAXIMUM value, or the maximum size of an unsigned 32-bit integer 4,294,967,295.

The client stores a current key index, an unsigned 32-bit value K_{idx} that is incremented each time a key is used by the client. The key index is initially set to zero, matching the starting index of the key array (and used as a 32 byte multiplier, dividing the larger array into 256-bit key-sized 'chunks').

Each time a key is consumed by the client, the key is erased and the counter is incremented, and the new index is sent in the next connection request to the branch server.

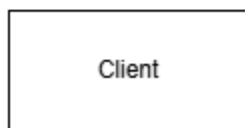
The client also has the branch server's long-term server secret ST_c embedded in its state, this secret is never sent across the network, but is used to verify the server to the client during the connection response stage of the tunnel formation.

The branch server generates the client ID, assigns the epoch and service class, sets the expiration time, and generates the client key tree for each client.



The branch generating the key tree by hashing the client identity string with the monotonic key counter for each key in the tree.

$$ID_{c,i} \leftarrow \{D_{dom}, D_{br}, epoch, Sc, C_{id}\}$$



$$K_{c,i} \leftarrow SHAKE256(K_{br} || ID_{c,i})$$

$$Exp \leftarrow Br_{exp}$$

The client also has the branch server's long-term server secret ST_c embedded in its state. This secret is never sent across the network. It is used with the active device leaf and the client nonce to validate the server during the connection response stage.

Once the expiration time is set, the client identity is assigned and the key-tree has been generated by the branch server, the client is ready to connect to a branch server.

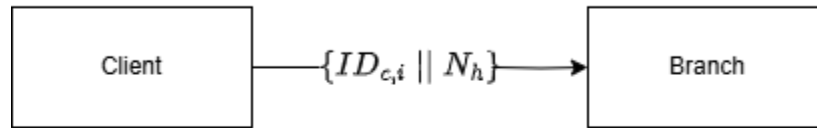
5.2.4 Client Connect Request

The client generates a per-session random nonce value N_h . The nonce and the client's key identity, consisting of the full root, branch, epoch, service, device, and key-index string, are sent to the branch server in a plaintext connection request. The connection request message body is $KID \parallel N_h$.

The client then uses cSHAKE-256 to derive the transmit and receive channel keys and nonces from the selected device leaf $K_{c,i}$, the protocol configuration string, and N_h . The transmit and receive channel symmetric cipher instances are initialized with the derived values.

The client generates a new session nonce: $N_h \leftarrow G(n)$

The client sends the server its identity string $ID_{c,i}$ and a random nonce N_h .



The client selects the next available key from the key tree:

$$K_{c,i} \leftarrow K_{tree}\{K_{c,i}\}$$

The client hashes the nonce and key to create the session keys:

$$Rk, Rn, Tk, Tn \leftarrow SHAKE256(K_{c,i} \parallel N_h)$$

The client keys the session receive and transmit ciphers

$$cpr_{rx}(Rk, Rn), cpr_{tx}(Tk, Tn)$$

The client stores a hash of the nonce, the base key and the servers secret:

$$H_c \leftarrow SHAKE256(N_h \parallel K_{c,i} \parallel ST_c)$$

The client computes a server-validation hash H_c from N_h , $K_{c,i}$, and ST_c using the implemented cSHAKE-256 input mapping. This value is stored temporarily and is used to verify that the server can regenerate the active device leaf and possesses the long-term server secret.

5.2.5 Server Connect Response

The server uses the received KID to regenerate the selected device leaf by applying cSHAKE-256 to the server derivation key and KID. The server then derives the same channel keys and nonces from $K_{c,i}$, SATP_CONFIG_STRING, and N_h , with transmit and receive directions inverted relative to the client.

The server derives the client key by hashing the branch key and the client key id:

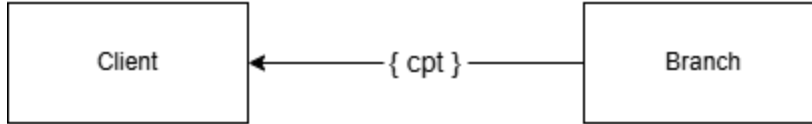
$$K_{c,i} \leftarrow \text{SHAKE256}(K_{br} || ID_{c,i})$$

The server hashes the nonce and key to create the session keys:

$$Rk, Rn, Tk, Tn \leftarrow \text{SHAKE256}(K_{c,i} || N_h)$$

The server keys the session receive and transmit ciphers

$$cpr_{rx}(Rk, Rn), cpr_{tx}(Tk, Tn)$$



The server calculates the session hash of the nonce, the base key and the servers secret:

$$H_c \leftarrow \text{SHAKE256}(N_h || K_{c,i} || ST_c)$$

The server encrypts and MACs the hash and sends it to the client:

$$cpt \leftarrow E_k(H_c, Ts || Seq)$$

The branch server computes H_c from N_h , $K_{c,i}$, and ST_c using the same cSHAKE-256 input mapping as the client. The server encrypts and authenticates H_c under the server transmit cipher with the serialized response header supplied as associated data, then sends the connection response to the client.

5.2.6 Tunnel Establishment

The client decrypts the connection response and compares the resulting value with the locally stored H_c . If the values are equal, the client has validated that the responder possesses ST_c and can regenerate the active device leaf. If the values are not equal, the client sends an authentication failure error, terminates the connection, and erases session state.

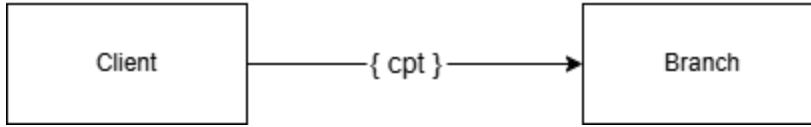
After the connection response is verified, the SATP tunnel cipher state is active. Client authentication is then performed over the encrypted tunnel. SATP v1.0a does not define an active keep-alive exchange.

5.2.7 Client Authentication Request

The client authentication request is carried as an encrypted tunnel message. The supplied client application prompts for the 32-byte passphrase token generated by the server, stores it in the device-key structure through the spass pointer, and the SATP client code sends $DID || token$ over the encrypted channel. The message is protected by RCS and the serialized SATP header is used as associated data.

The client is prompted for a password which is hashed:
 $H_p \leftarrow \text{SHAKE256}(\text{pass})$

The client encrypts the hash and adds the sequence number and timestamp to the data AEAD and sends it to the sever:
 $\text{cpt} \leftarrow E_k(H_p, Ts \parallel Seq)$



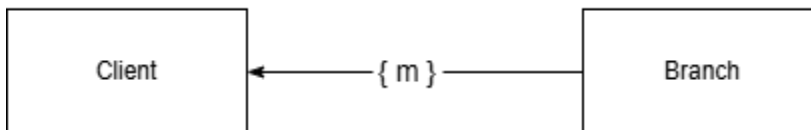
5.2.8 Server Authentication Response

The server receives the first encrypted tunnel message and passes the decrypted DID || token to the application-supplied authentication callback. The supplied server application fetches the stored credential associated with the DID and verifies the received token with `satp_server_passphrase_hash_verify`. The stored credential is generated by applying SCB to the 32-byte server-generated token during provisioning. If verification succeeds, the server sends the encrypted 4-byte SID to the client. If verification fails, the server closes the connection and erases session state.

The server decrypts the authentication token: $U_n, H_p \leftarrow -E_k(\text{cpt})$

The server has an SCB hash of the clients passphrase stored, hashes the passphrase hash and compares it to the stored value:
 $\text{stm} \leftarrow \text{SCB}(H_p)$

The server verifies the password hash, and sends the pass/fail authentication response back to the client.



5.2.9 Client Authentication Verification

Upon receiving the authentication response, the client decrypts the response and verifies that the returned SID matches the SID prefix embedded in the device KID. If the check succeeds, the client regards the tunnel as authenticated and may exchange application data. If the response is absent, malformed, or incorrect, the connection is closed and the session state is erased.

5.2.10 Packet Processing and State Requirements

SATP endpoints shall reject encrypted packets whose sequence value is not exactly one greater than the receiver sequence counter. The receiver sequence counter is advanced only after authenticated decryption succeeds. This strict ordering rule prevents replay within an established tunnel, but it does not tolerate packet reordering.

SATP endpoints shall reject packets whose timestamp is outside SATP_PACKET_TIME_THRESHOLD. The reference implementation defines SATP_PACKET_TIME_THRESHOLD as 60 seconds. This check depends on wall-clock synchronization within the configured threshold.

SATP endpoints shall supply the serialized 21-byte packet header as AEAD associated data before encrypting or decrypting encrypted packets. Header modification therefore causes authenticated decryption failure.

The server does not store the full client key tree. It regenerates a device leaf from the server derivation key and received KID. Deployments requiring server-side duplicate-leaf rejection shall maintain persistent consumed-index state outside the basic key derivation function.

6. Mathematical Description

Mathematical Symbols

$\leftarrow \leftrightarrow \rightarrow$	-Assignment and direction symbols
$:=, !=, ?=$	-Equality operators; assign, not equals, evaluate
C	-The client host, initiates the exchange
S	-The server host, listens for a connection
Ad	-The AEAD additional data
cfgs	-The protocol configuration string
C_{ki}	-The current client key index number
cpr_{rx}	-The receive channel cipher
cpr_{tx}	- The transmit channel cipher
cpt	- The output ciphertext
D_{br}	-The branch directory identity string
D_{dom}	-The domain directory identity string
D_{id}	-The device identity string
Ep_c	-The epoch class string
H_c	-The cSHAKE-derived server-validation hash computed from N_h , $K_{c,i}$, and ST_c
H_p	-The 32-byte passphrase token supplied by the client application
$ID_{c,i}$	-The client and key identity string
$E_k, -E_k$	-Symmetric encryption and decryption
$K_{c,i}$	-The secret client key at index i
K_{tree}	-The user's tree of symmetric keys
K_{root}	-The master root key
N_h	-The per session nonce
R_k	-The receive channel session key
R_n	-The receive channel nonce
SCB	-SHAKE Cost Based KDF
S_{id}	- The service identity string
S_p	-The passphrase token hash

ST_c	-The long-term server secret
T_k	- The transmit channel session key
T_n	-The transmit channel nonce
t	-The tag length
T_{auth}	-A hash of the base session key, the server salt and shared secret
U_n	-The user name
U_t	-The user token

6.1 Domain Initialization

Network initialization begins by generating a master secret K_{root} , and a 16-bit domain identity string D_{dom} .

$$K_{root} \leftarrow G(n)$$

$$D_{dom} \leftarrow (D_{id})$$

6.2 Branch Initialization

Using the master domain key, the branch keys are created by hashing the master domain key and the 16-bit branch domain identity string and 16-bit branch identity string.

Every branch-master key is created by:

$$K_{br} \leftarrow \text{cSHAKE256}(\text{key} = K_{root}, \text{name} = \text{cfigs}, \text{custom} = D_{dom} \parallel D_{br}).$$

Every branch server retains a common branch-server secret, installed on every client and used to authenticate the server to the client in the first encrypted message that tests the tunnel. This branch secret is transferred to every client the server creates.

$$ST_c \leftarrow G(n)$$

6.3 Device Initialization

The user's identity string consists of the domain identity (D_{dom}), the branch identity (D_{br}), the Epoch Class (Ep_c), Service identity (S_{id}), the user identity (U_{id}) and Key identity $ID_{(c,i)}$.

The server generates key batches by hashing the identity string, and a monotonic counter.

For every key index i ($i, i^{+1}, \dots n$) the manufacturer pre-computes:

$$K_{c,i} \leftarrow \text{cSHAKE256}(\text{key} = K_{br}, \text{name} = \text{empty}, \text{custom} = ID_{c,i})$$

and stores $\langle \text{ID}_{c,i}, \text{K}_{c,i} \rangle$ on the storage device. The reference implementation generates 1024 leaves per device key record. The key identity includes a monotonic counter stored in the final four bytes of the KID, and the current implementation accepts only counters lower than $\text{SATP_KEY_TREE_COUNT}$.

The client stores the long-term server secret token ST_c , used to validate the server during the authentication phase.

When the user first joins the network, through a separate registration process, the server generates a 32-byte printable passphrase token and associates it with the device identity. The supplied server application stores $\text{SCB}(\text{token})$ in a user credential record indexed by DID, and the supplied client application later presents the token over the encrypted tunnel.

$\text{Hp} \leftarrow \text{token}$, where $|\text{token}| = \text{SATP_HASH_SIZE}$

$\text{Sp} \leftarrow \text{SCB}(\lambda, \text{Hp})$

$\text{Ut} \leftarrow \{ \text{Did}, \text{Sp} \}$

The server stores the device identity and SCB-hardened passphrase token in its credential database. The token is not transmitted during registration by the SATP tunnel code; registration is a provisioning operation performed by the server application or an external enrollment process.

$\text{Did}, \text{Sp} \leftarrow \text{Ut}$

Verify record association by DID

Store Ut for later callback-mediated authentication

6.4 Connection Request

The client selects a fresh key from the key-tree (K_{tree}) at the corresponding current index number (C_{ki}). The counter is incremented as soon as a key is read, and the value is stored on the client memory device.

$\text{K}_{c,i} \leftarrow \text{K}_{\text{tree}}\{ \text{K}_{\text{idx}} \}$

The user generates a session nonce (N_h), combines this with the protocol configuration string, and the base key, and derives the receive and transmit channel session keys and nonces, and increments the internal key index.

$\text{N}_h \leftarrow \text{G}(n)$

$\text{Rk}, \text{Rn}, \text{Tk}, \text{Tn} \leftarrow \text{cSHAKE256}(\text{key} = \text{K}_{c,i}, \text{name} = \text{cfigs}, \text{custom} = \text{N}_h)$

$\text{K}_{\text{idx}} = \text{K}_{\text{idx}} + 1$

The client sends the server its identity string including the current key index and the random nonce.

$$C\{ID(c,i), N_h\} \rightarrow S$$

Once a key is pulled from the key pool and used to key the set of receive and transmit cipher instances, the key is permanently erased from the key pool (whether the connection succeeds or not).

The client initializes the transmit and receive cipher instances and waits for a response from the server.

$$\begin{aligned} & \text{cpr}_{\text{rx}}(R_k, R_n) \\ & \text{cpr}_{\text{tx}}(T_k, T_n) \end{aligned}$$

The client hashes the session nonce (N_h), the session base key ($K_{c,i}$), and the server's long-term secret (ST_c) and stores the hash.

$$H_c \leftarrow \text{cSHAKE256}(\text{key} = N_h, \text{name} = K_{c,i}, \text{custom} = ST_c)$$

6.5 Connection Response

The server uses the client's key identity string to regenerate the selected tree-key:

$$K_{c,i} \leftarrow \text{cSHAKE256}(\text{key} = K_{br}, \text{name} = \text{empty}, \text{custom} = ID_{c,i})$$

The server creates the transmit and receive cipher keys and nonces by hashing the session nonce, the protocol configuration string, and the user key.

$$R_k, R_n, T_k, T_n \leftarrow \text{cSHAKE256}(\text{key} = K_{c,i}, \text{name} = \text{cfigs}, \text{custom} = N_h)$$

The server creates the session validation hash by hashing the session nonce (N_h), the session base key ($K_{c,i}$), and the server's long-term secret (ST_c).

$$H_c \leftarrow \text{cSHAKE256}(\text{key} = N_h, \text{name} = K_{c,i}, \text{custom} = ST_c)$$

The server keys the transmit and receive cipher instances:

$$\begin{aligned} & \text{cpr}_{\text{rx}}(R_k, R_n) \\ & \text{cpr}_{\text{tx}}(T_k, T_n) \end{aligned}$$

The server encrypts and MACs the session validation hash, adding the packet timestamp and sequence number to the AEAD data, and sends it to the client:

$$cpt \leftarrow E_k(H_c, Ts \parallel \text{Seq})$$

$$S\{cpt\} \rightarrow C$$

6.6 Tunnel Establishment

The client adds the packet sequence and timestamp to the AEAD data, and verifies and decrypts the session authentication hash:

$$h \leftarrow -E_k(cpt, Ts \parallel Seq)$$

The client compares the hash to the message, and if the hash values are equivalent, the tunnel has been raised successfully.

$$\text{Verify}(h, H_c) \rightarrow \{ \text{true}, \text{false} \}$$

6.7 User Authentication

Once the tunnel has been established, the client supplies the passphrase token to the SATP client code. The token is encrypted and sent to the server over the established tunnel with the packet header bound as AEAD associated data. The client waits for the server's authentication response.

$$Hp \leftarrow \text{token, where } |Hp| = \text{SATP_HASH_SIZE}$$

$$cpt \leftarrow E_k(\text{Did} \parallel Hp, \text{header})$$

$$C\{cpt\} \rightarrow S$$

6.8 Authentication Response

The server verifies and decrypts the user authentication token. The device identity string is sent with the passphrase token and is used by the server authentication callback to look up the stored SCB-hardened credential.

$$\text{Did}, Hp \leftarrow -E_k(cpt, \text{header})$$

The server fetches the user token containing the hashed passphrase token, for comparison with the passphrase token sent by the client.

$$U_t \leftarrow \{ U_n, S_p \}$$

The server runs the SCB verifier on the received token and compares the result with the stored credential associated with the DID.

$$S_{tmp} \leftarrow \text{SCB}(\lambda, Hp)$$

$$\text{Verify}(S_{tmp}, S_p) \rightarrow \{ \text{true}, \text{false} \}$$

If the token matches the stored value, the server sends an authentication success response containing the encrypted SID. If the challenge fails, the server sends an authentication failure message when possible, closes the connection, and erases the session state.

$$cpt \leftarrow Ek(Sid, \text{header})$$
$$S\{cpt\} \rightarrow C$$

6.9 Authentication Verify

The client receives the response from the server. If the response contains the encrypted server identity string and the decrypted SID matches the SID prefix embedded in the device KID, authentication succeeds. If the message fails authentication, contains an unexpected value, or is absent, the client tears down its side of the tunnel and erases session state.

$$Sid \leftarrow -Ek(cpt, Ts \parallel Seq)$$

7. Security Analysis

Scope of this section: We analyze SATP at three concentric layers:

1. **Primitive layer** (RCS cipher, SHAKE/cSHAKE, KMAC, SCB-KDF)
2. **Protocol layer** (key hierarchy, handshake, tunnel, replay controls)
3. **System layer** (deployment, compromise & recovery, side-channels)

All cost metrics assume the best public attacks as of *July 2025* and a quantum adversary limited to Grover-style square-root searches (no full hidden-shift for large-round Keccak).

7.1 Adversary Model

Capability	Assumed?	Notes
Full packet capture & injection	Yes	Standard IND-CCA setting; adversary observes and modifies traffic.
Compromise of <i>one</i> branch server (Kbr)	Yes	Models supply-chain or local intrusion.
Compromise of <i>one</i> client device	Yes	Attacker recovers a single unused Kc,i.
Quantum computer (Grover)	Yes	
Large-scale hidden-shift over Keccak	No current feasibility	24-round permutation exceeds known breakpoints
Side-channel (timing/power)	Bounded	Constant-time references; leakage-free RNG assumed.

Security goals: authenticated confidentiality for tunnel traffic, ciphertext integrity, server authentication during tunnel establishment, callback-mediated client authentication after tunnel establishment, strict in-order replay rejection for established encrypted packets, and compromise containment for consumed client leaf keys under the stated deployment assumptions.

7.2 Primitive Strength

Primitive	Parameter	Classical Security	Quantum Security	Commentary
RCS-256 (wide-block Rijndael + 22 rnd)	256-bit key, 256-bit block	Best structural attack $\geq 2^{254}$ ops; no distinguisher on full rounds	Grover $\Rightarrow 2^{128}$ ops	Wide-block doubles birthday bound; cSHAKE schedule defeats related-key rectangles.
KMAC-256	256-bit capacity	IND-CPA / SUF-CMA bound 2^{-128}	Collapsing $\Rightarrow$ post-quantum INT-CTXT	Immune to Simon attacks that break GHASH/Poly1305.
SCB-KDF	Deployment-selected	Cost increases with	Quantum parallelism	Used by the supplied authentication

	cpucost and memcost; reference defaults are 1 and 1 MiB	configured CPU and memory parameters	remains constrained by memory cost and verifier configuration	callback to verify a 32-byte passphrase token.
SHAKE-256 / cSHAKE-256	256-bit capacity	Pre-image $\geq 2^{256}$	Grover $\geq 2^{128}$	Sponge with full-capacity security.

The primitive security margins are intended to exceed 128-bit post-quantum work factors under the stated assumptions. Protocol-level assurances additionally depend on correct provisioning, non-reuse of device leaves, client nonce uniqueness, and protection of STc and Kbr.

7.3 Confidentiality

1. **Session-level secrecy** - Per-session keys are derived via cSHAKE256(key = Kc,i, name = cfgs, custom = Nh). Breaking confidentiality reduces to compromise of Kc,i for that session, failure of nonce or leaf uniqueness, compromise of STc or Kbr sufficient to regenerate active material, or a break of the underlying RCS/cSHAKE assumptions.
 - Recover Kc,i (pre-image $2^{256} \rightarrow 2^{128}$ under Grover) **or**
 - Collide SHAKE256 outputs (birthday 2^{128}) **or**
 - Break RCS-256 under known-key ($\geq 2^{254}$).
2. **Past-session protection** - Kc,i is erased after first use by the client-side provisioning record update. Compromise of future unused leaves gives no direct oracle on past traffic when prior leaves were consumed and erased and the long-term branch/server secrets were not available to the adversary at the time of capture.
3. **Branch compromise containment** - Exposure of Kbr allows derivation of device leaves for that branch. Past sessions remain protected only if the relevant consumed Kc,i values and session nonces cannot be reconstructed from retained device records or captured state.

7.4 Integrity & Authentication

- **Packet integrity** – Encrypt-then-KMAC yields INT-CTXT with forging probability $\leq 2^{-128}$ per packet.
- **Server authentication** - The client verifies Hc = cSHAKE256(key = Nh, name = Kc,i, custom = STc). Only a server possessing STc and able to regenerate the correct Kc,i can produce the encrypted validation value under the derived channel state.
- **Client authentication** - The client sends a 32-byte passphrase token over the encrypted tunnel. The supplied server application verifies this token through a callback using the SATP SCB verifier and a stored SCB-hardened credential. The default SCB cost constants are implementation defaults and may be raised by a deployment profile.

7.5 Forward Secrecy

Secret Leaked	Affects	Unaffected
Single Kc,i	That session if the leaf is active or recoverable	Other sessions whose leaves remain secret and were not reused
Branch Kbr	Future sessions of that branch and any reconstructible sessions	Past sessions whose consumed leaves and session state cannot be reconstructed
Kroot	Entire domain future	Past sessions survive if all Kc,i already consumed

Epoch roll and reprovisioning allow a domain to invalidate future key use after a branch or root secret is replaced. The speed and scale of such operations are deployment properties rather than wire-protocol guarantees.

7.6 Replay, Re-ordering & DoS

- A 64-bit UTC timestamp and a 64-bit sequence number are authenticated as AAD for encrypted packets. Packets outside the timestamp window or not exactly equal to the next expected sequence value are rejected.
- The reference implementation uses strict in-order sequence processing. It does not implement a 32-packet sliding replay window.
- Malformed or forged encrypted packets are rejected after header validation and authenticated decryption failure. The receiver sequence counter is advanced only after successful authentication.

7.7 Side-Channel & Fault Resistance

- **Constant-time** RCS reference avoids S-box tables; AES-NI path is data-independent.
- **SCB** scattering obscures memory-access patterns, diminishing cache-timing leakage on passphrase derivation.
- **Fault detection** – RCS final-row checksum ($\text{prob} \geq 1-2^{-15}$) aborts on single-byte glitches; higher-order countermeasures possible via redundant MAC.

7.8 Compositional Security Proof Sketch

1. **Primitives:** Assume RCS is a QIND-CPA PRP; KMAC is QSUF-CMA.
2. **Tunnel:** Encrypt-then-MAC (Bellare–Namprempre) $\Rightarrow$ IND-CCA & INT-CTXT.
3. **Handshake:** The implemented tunnel establishment provides implicit server authentication through Hc. Client authentication is performed after tunnel establishment by the application authentication callback.
4. **Overall:** Under the stated assumptions, SATP provides authenticated encrypted transport for established tunnel messages. Stronger AKE claims require an explicit transcript model, duplicate-leaf rejection policy, and proof obligations that are outside the current reference implementation.

7.9 Residual Risks & Mitigations

Risk	Mitigation
Nonce reuse (client power loss before Kidx++ write)	Atomic key-file update or monotonic counter in secure element; deployments requiring stronger replay control should maintain server-side consumed-index state.
Weak RNG for Nh	Feed hardware TRNG; fallback to DRBG reseeded per 32 KiB.
Physical extraction of STc	Store STc in tamper-resistant SE; treat leak as branch-level compromise and rotate epoch.
Side-channel on SCB	Mask scatter indices; use temperature-stabilized DRAM to blur power patterns.

7.10 Comparison with Competing Post-Quantum Schemes

Scheme	Crypto Type	HW cost (server)	PQ level	FS	Replay	Notes
SATP	Symmetric-only	SHA-3 + RCS (~5 KB code)	Cat-V	Bounde d	Strict +1	Fixed 16-byte KID, no PKI, post-tunnel callback authentication
NIST Kyber + TLS 1.3	PQ KEM + AEAD	≈200 KB code + certs	Cat-I / III	✓	✓	Heavy handshake; cert ops
HPKE (FrodoKEM)	PQ KEM + symmetric	>500 KB code	Cat-V	✓	✓	10 × slower, 4 KB messages
MatrixVPN (LWE)	LWE KEX + AES-GCM	256 KB code	Cat-II I	✓	Partial	GHASH 64-bit Q tag

SATP attains comparable or higher quantum security with ~10× smaller code footprint and no reliance on CA infrastructure.

8. Real-World Use-Case Scenarios

SATP's symmetric-only, post-quantum design supports a wide array of deployment models that benefit from low handshake latency, minimal server state, and deterministic provisioning costs.

8.1 FinTech: Instant Low-Value Payments

Context. Contactless and mobile payments under \$20 are latency-sensitive and often processed offline (transit, vending, pop-up retail).

- SATP smart-cards embed thousands of single-use indices; terminals approve locally using a single SHAKE hash.
- Back-end settlement reconciles spent indices nightly, rotating the branch epoch to revoke lost or stolen cards.
- **Benefit:** reduced local authorization latency by avoiding per-transaction certificate validation in deployments where SATP provisioning is already in place.

8.2 Enterprise Network Login & Zero-Trust Segmentation

Context. Modern zero-trust architectures require mutual authentication for every internal service call, stressing PKI infrastructure.

- Workstations and servers store SATP identity strings in firmware TPM or secure elements.
- SATP can replace certificate-chain establishment for selected internal service calls where symmetric provisioning and callback-mediated authentication are acceptable.
- **Benefit:** reduced certificate-management overhead and faster re-authentication in controlled environments.

8.3 IoT & Edge Devices (Smart Grid / Sensors)

Context. Millions of constrained devices need long-term security yet cannot afford PQ public-key overhead.

- Each meter receives a key-tree at manufacture; a substation holds its branch key only.
- Outage-proof: devices authenticate even if the WAN link to the CA is offline.
- **Benefit:** lower handshake energy than public-key exchange when pre-provisioned key trees are available; branch compromise remains bounded by branch-key rotation and reprovisioning policy.

8.4 Critical Infrastructure & SCADA

Context. PLCs and RTUs in power and water systems operate for decades with scarce firmware head-room.

- SATP uses a compact symmetric code path and does not require RSA or ECC libraries in the tunnel establishment path.
- Operators rotate epochs by swapping a USB token during maintenance, re-keying an entire plant in minutes.
- **Benefit:** Standards-compliant quantum security without disrupting legacy field-bus latency budgets.

8.5 Secure Remote & Rural Banking (Offline Cash)

Context. Rural agents dispense cash where connectivity is intermittent.

- SATP cards preload daily withdrawal quota; agents verify offline and sync indices when online.
- **Benefit:** Eliminates desktop PKI hardware; lowers cash-out fraud window to the unspent index range.

8.6 Healthcare Devices & Body-Area Networks

Context. Pacemakers and insulin pumps need authenticated telemetry with minimal power draw.

- Tokens provision ~1 000 SATP indices; a doctor's reader validates in <1 ms with zero public-key handshake.
- **Benefit:** lower handshake overhead for constrained medical devices when provisioning and audit controls satisfy the applicable regulatory environment.

8.7 Post-Disaster Mesh & Humanitarian Relief

Context. In disaster zones, infrastructure-less radios must exchange situational data securely.

- 16-byte SATP IDs fit inside LoRa frames; timestamp windowing rejects replay even when clocks drift.
- **Benefit:** Entire enclave remains secure for weeks with no external CA or internet.

8.8 Satellite & Space Communications

Context. Small satellites require deterministic crypto budgets and long operational life.

- Each CubeSat can carry a provisioned set of SATP indices sized to its operational telemetry policy.
- Ground stations roll branch keys at launch, never needing certificate uplinks.
- **Benefit:** Predictable CPU cycles, radiation-hardened symmetric code only.

8.9 Manufacturing & OT Network Segments

Context. Factory robots and AGVs authenticate controller commands on millisecond schedules.

- SATP timestamps and sequence numbers are MAC'd as AAD, stopping replay without PLC time-sync.
- **Benefit:** reduced handshake overhead compared with certificate-based overlays in controlled OT segments, while keeping symmetric post-quantum safety margins.

8.10 Media & DRM Micropayments

Context. Streaming services monetize per-view content; traditional DRM adds heavy overhead.

- Each SATP index equals one content license; the player hashes and burns an index to unlock playback.
- **Benefit:** Sub-millisecond validation, fixed operational cost, privacy via rotating identity strings.

Conclusion

SATP demonstrates that a symmetric-first, hash-driven security stack can provide authenticated encrypted transport for environments where deterministic provisioning and low handshake cost are preferred over online public-key establishment. Key take-aways include:

1. **Post-Quantum-Oriented Symmetric Core:** RCS-256, SHAKE/cSHAKE-256, KMAC-256, and SCB-KDF provide a symmetric cryptographic foundation whose brute-force security margins are not affected by Shor-style attacks against public-key systems.
2. **Minimal Server Burden:** A branch server stores a compact server key structure and regenerates device leaves on demand. It does not store complete client key trees in the reference implementation.
3. **Single-Use Leaf Separation:** Every session is intended to consume one client leaf. Correct compromise containment depends on atomic client key-state updates, secure erasure of consumed leaves, and protection of STc and Kbr.
4. **Replay-Resistant Established Transport:** Timestamp and sequence fields are authenticated as AAD for encrypted packets. The implementation enforces strict in-order replay rejection rather than a sliding replay window.
5. **Deployment Flexibility:** The 16-byte KID supports domain, branch, epoch, service, device, and key-index fields, allowing SATP deployments to separate administrative domains and service classes within a compact identity string.

Strategic Outlook (2025-2030)

- **FinTech:** Expect pilot roll-outs in micro-payment and offline cash sectors within 18 months, driven by dramatic cuts in tap latency and CA overhead.
- **Critical Infrastructure:** Utilities are poised to retrofit SATP during scheduled firmware updates, replacing ageing RSA stacks with <32 kB symmetric code.
- **IoT & Edge:** Vendors of smart-grid meters and healthcare wearables plan to embed SATP key-trees at manufacture, eliminating field PKI provisioning.
- **RegTech & Audit:** Financial institutions foresee SATP tunnels as tamper-evident log channels, slashing PKI certificate lifecycle costs.

Research & Development Roadmap

1. **Optional Hybrid Extension:** Define an optional post-quantum KEM-assisted profile for deployments that require asymmetric freshness in addition to symmetric provisioning.
2. **Automated Epoch Services:** Define a standard RESTful endpoint for secure epoch-bump broadcasting and mass revocation.
3. **Formal Verification:** Complete mechanized proofs in Tamarin or ProVerif for the implemented two-message tunnel establishment, post-tunnel authentication callback model, and strict in-order transport semantics.
4. **Side-Channel Hardening Kit:** Publish reference masking and fault-detection wrappers for RCS and SCB to ease certification (FIPS 140-4, Common Criteria).

SATP is a practical symmetric tunnel construction for environments where long-life assets and quantum threat models collide. Its assurance depends on disciplined provisioning, reliable single-use key-state advancement, authenticated transport processing, and accurate conformance testing against the implemented wire format.